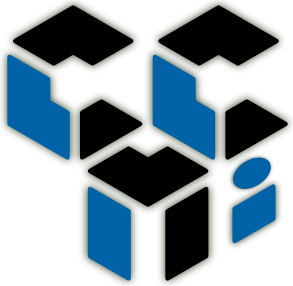
# Object Constraint Language





FAKULTA
INFORMAČNÍCH
TECHNOLOGIÍ
ČVUT V PRAZE

**BI-KOM: Konceptuální modelování (Cvičení #7)**

Zimní semestr 2026/2027 (B261)

# Plán 7. cvičení

1. Shrnutí aktivity z minula
2. Modelování omezení s OCL
3. Příklad na procvičení (za bod)
4. Úkoly na příště

# Shrnutí aktivity z minula

- Neodstranění všech chyb
- Person je typický Kind
- Role potřebují pečetidlo
- Neopravení patternu DepPhase (relačně závislá phase)
- Myšlenka "Prázdný model nemá chyby" -> k čemu nám potom je?

# OCL

- Jazyk pro specifikaci omezení, určen primárně pro UML model
- Můžeme použít i pro OntoUML, protože je to UML profil
- Vždy se specifikuje `context`, ke kterému se omezení vztahuje
- Občas lze omezení specifikovat z pohledu více kontextů, je vhodné zvolit takový, odkud je specifikace nejsnazší (např. u vztahů a smyček)
- Základem jsou booleovské výrazy vždy s výsledkem `true` nebo `false`
- Omezení v OCL pak mohou sloužit k validaci instancí příslušného modelu
- Existují i metody jak použít OCL v kódu či testech

# OCL: `inv`ariant

- Omezení, které platí ve všech situacích

```
context AdultPerson
inv isAdult: self.age >= 18

context Token
inv camelCaseName: name.matches("[A-Z][A-Za-z]*")

context Person
inv notOwnKid: not kids->includes(self)
```

- Je vhodné si omezení pojmenovávat (pro identifikaci)
- Implicitní `self` dle kontextu (je-li jednoznačné)
- Rozdíl mezi `.` a `->`

# OCL: `precondition`

- Podmínky pro volání metody
- Co musí platit před tím, než se metoda zavolá

```
context Person::die()
pre isAlive: self.deathdate = null

context Stack::pop()
pre: not self.isEmpty()

context Integer::divideBy(x:Integer)
pre: x.value <> 0
```

- Pozor na operátory `=` a `<>`

# OCL: `postcondition`

- Podmínky po volání metody
- Co musí platit po tom, co se metoda zavolá
- Společně s `precondition` představuje *kontrakt metody*

```
context Person::die()
post isNotAlive: self.deathdate <> null

context Stack::pop()
post: size = size@pre – 1

context Integer::divideBy(x:Integer):Integer
post: result = self.value / x.value
```

- `@pre` odkazuje na hodnotu před voláním
- `result` představuje výsledek operace, `=` **není** přiřazení

# OCL: `init` a `derive`

- `init` umožňuje kontrolovat nastavení výchozích hodnot při vzniku objektu

```
context Stack
init: isEmpty()

context Counter
init: self.value = 0
```

---

- `derive` slouží k odvození atributů (resp. vztahů)

```
context Order::totalPrice:Integer
derive: self.items->collect(i | i.price)->sum()
```

# OCL: výrazy

- Booleovské operace: `and`, `or`, `not`, `xor`, `implies`, `if then else`
- Základní typy a operace s nimi, vlastní typy, porovnávání
- Omezení `pre`, `post`, `init`, `inv`, `derive`, `def`, `body`
- `let in` (lokální proměnná v podmínce), `package` / `endpackage`, `import`
- Kolekce: transformace, dotazování, iterace, množinové operace

```
context EventForSexists
def: allMen:Boolean = self.participants->forAll(p | p.isMan())
def: allWomen:Boolean = self.participants->forAll(p | p.isWoman())
inv: allMen or allWomen
```

# OCL: `implies` vs `if then else`

- `if then else` je často zbytečné a booleovský výraz lze zapsat bez něj
- OCL není imperativní programování, `if then else` je výraz, kde musí vždy být obě větve a mít stejný typ
- `if A then B else true` je to samé jako `A implies B`
- `if A then true else B` je to samé jako `A or B`
- `if A then true else false` je to samé jako `A`
- `if A then false else true` je to samé jako `not A`
- (toto se překvapivě často objevuje i ve zdrojových kódech 🤦)

# OCL: kolekce a operace

- Použití kolekcí a operací (případně další datové typy) je žádoucí
- Nelze si ale *vymýšlet* vlastní operace a typy/kolekce
- Existují různé odlišnosti dle implementace OCL (v různých nástrojích)
- Ideálně používejte Acceleo OCL dokumentaci
- V ostatních případech uveďte referenci a buďte konzistentní

# OCL: reference a upozornění

- Acceleo OCL
- OCL specifikace
- Introduction to OCL (Universität Koblenz-Landau)
- OCL: A Definitive Guide
- Introduction to OCL (SFM'12 Summer School)

---

- V OCL specifikujte pouze to co **nelze** zachytit přímo v (Onto)UML
- Někdy stačí:
  - Vytvořit hierarchii podtříd
  - Přesunout vztah/atribut do podtřídy
  - Zvolit správně multiplicity či definovaná omezení (`{complete}`, `{essential}`, ...)

# Na co si dát v semestrální práci pozor

- Textový popis a OntoUML model musí být konzistentní
- K OntoUML modelu musíte mít tabulku použitých konstruktů
- OCL omezení musí mít 3 netriviální dle popisu + alespoň 2 další
- Pokud je OntoUML model složitý a méně přehledný, rozdělte jej vhodně do více diagramů
- Konzultujte svou práci, ideálně během cvičení ale případně i mimo něj

# Na co si dát v semestrální práci pozor

- Nelze mít asociace ani třídy bez stereotypů
- Každá instance musí mít právě jednu identitu
- Pravidla pro dědičnosti, omezení `{disjoint}` a `{complete}`
- Relační závislost pro «Role» a «RoleMixin»
- Správné multiplicity dle definic vztahů (např. «characterization» nebo «subQuantityOf»)
- Pravidla pro použití omezení a typů vztahů celek-část
- Konzistentní multiplicity u materiálních vztahů
- Dodržování notace UML (/ = derived, *abstract*, ...)
- Dodržení syntaxe OCL (`if-then-else`, `implies`, `context`, ...)
- ...

# Aktivita za bod

- OntoUML model: Taxislužba
- Zadaný model (ke stažení na Moodle):

1. V souladu se zadaným modelem pomocí OCL zapište zadaná 4 omezení
2. Vymyslete alespoň 1 další omezení pro zadaný model

---

Odevzdejte (ideálně jako text přímo do Moodle) v: **Aktivita / Activity #7**

# Úkoly do příště

- Finalizovat semestrální práci:
  - Doladit textový popis a OntoUML model
  - Specifikovat omezení pomocí OCL
- Finalizovat semestrální práci a zkontrolovat si ji
- Případně konzultovat před nebo na příštím cvičení
- Příští cvičení: konzultace semestrálních prací
  - Máte možnost ukázat svoji semestrální práci ...
  - ... a dostat feedback, co opravit nebo zlepšit

# Konec prezentace